

An
Industry Oriented Mini Project Report
on
Autism spectrum disorder analysis for toddlers

Submitted in partial fulfillment of the requirements for the award of degree

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Submitted By

S.DARSHAN

227Z1A66B4

Under the Guidance of

Mrs. G. MANASA

Assistant Professor



SCHOOL OF ENGINEERING

Department of Computer Science and Engineering

(Artificial intelligence and machine learning)

NALLA NARASIMHA REDDY
EDUCATION SOCIETY'S GROUP OF INSTITUTIONS
(AN AUTONOMOUS INSTITUTION)

**Approved by AICTE, New Delhi, Chowdariguda (V) Korremula 'x' Roads,
via Narapally, Ghatkesar (Mandal) Medchal (Dist), Telangana-500088**

2024-2025

SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report titled **“AUTISM SPECTRUM DISORDER ANALYSIS FOR TODDLERS”** is being submitted by **S.Darshan (227Z1A66B4)** in Partial fulfillment for the award of **Bachelor of technology in Computer Science & Engineering(Artificial intelligence and machine learning)** is a record bonafide work carried out by them. The results embodied in this report have not been submitted to any other University for the award of any degree.

Internal Guide

(Mrs.G.Manasa)

Head of the Department

(Dr.G.Sravan kumar)

Submitted for the University Examination held on.....

External Examiner

DECLARATION

I S.Darshan the students of **Bachelor of Technology in Computer Science and Engineering(Artificial intelligence and machine learning)**, **Nalla Narasimha Reddy Education Society's Group of Institutions**, Hyderabad, Telangana, hereby declare that the work presented in this project work entitled “**AUTISM SPECTRUM DISORDER ANALYSIS FOR TODDLERS**” is the outcome of our own bonafide work and is correct to the best of our knowledge and this work has been undertaken taking care of engineering ethics. It contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning.

S.DARSHAN

227Z1A66B4

Date:

Signature:

ACKNOWLEDGEMENT

I express our sincere gratitude to our guide **Mrs. G. Manasa**, Assistant Professor, Department of Computer Science and Engineering(**Artificial intelligence and machine learning**), NNRESGI, who motivated throughout the period of the project and also for her valuable and intellectual suggestions, guidance, and constant encouragement right throughout our work.

I would like to express our profound gratitude to our project coordinator **Mr. SAMUEL CHEPURI**, Assistant Professor, Department of Computer Science and Engineering(**Artificial intelligence and machine learning**), NNRESGI, for his support and guidance in completing our project and for giving us this opportunity to present the project work.

I profoundly express thanks to **Dr. G .SRAVAN KUMAR**, Professor & Head, Department of Computer Science and Engineering, NNRESGI, for his cooperation and encouragement in completing the project successfully.

I wish to express our sincere thanks to **Dr. G. Janardhana Raju**, Dean School of Engineering, NNRESGI, for providing the facilities for completion of the project.

I wish to express our sincere thanks to **Dr. C. V. Krishna Reddy**, Director, NNRESGI, for providing the facilities for completion of the project.

Finally, I would like to thank Project Review Committee (PRC) members, all the faculty members and supporting staff, Department of Computer Science and Engineering, NNRESGI, for extending their help in all circumstances.

By

S.DARSHAN

227Z1A66B4

ABSTRACT

Autism Spectrum Disorder (ASD) is a neuro-developmental condition that affects communication, social interaction, and behavior. Early detection plays a crucial role in improving outcomes through timely interventions. This project aims to develop a web-based autism screening tool that collects responses to 10 key behavioral questions and uses a trained machine learning (ML) model to assess ASD risk levels. Based on the responses, the system will classify the result as "Mild," "Severe," or "No ASD Traits" and provide precautionary recommendations. The project integrates data preprocessing, feature selection, and predictive modeling to enhance accuracy. A user-friendly interface will allow parents, caregivers, and clinicians to conduct preliminary assessments. The system leverages historical screening data to train an ML model, ensuring reliable predictions. The model will be optimized for accuracy and efficiency, enabling real-time responses on the web interface. Additionally, it will provide users with precautionary steps based on the severity of detected ASD traits, guiding them toward appropriate next steps, such as seeking professional evaluation. By combining technology with healthcare screening, this project aims to offer an accessible and effective early intervention tool for autism awareness and detection.

keywords: Autism Screening, Machine Learning, ASD Detection, Early Diagnosis, Behavioral Analysis, Expected Treatment Level.

TABLE OF CONTENTS

	Page No.
Abstract	i
List of Figures	ii
List of Tables	iii
List of Abbreviations	iV
1. INTRODUCTION	1
1.1 Motivation	1
1.2 Problem Definition	2
1.3 Objective of project	2
1.4 Limitation of project	3
1.5 Scope of project	3
1.6 Relevance in Current Healthcare Scenario	4
2. LITERATURE SURVEY	5
2.1 Introduction	5
2.2 Existing System	5
2.3 Proposed System	6
2.4 Advantages of Traditional Systems	7
2.5 Limitations of Traditional Systems	7
2.6 Advantages of AI-Based Systems	8
2.7 Limitations of AI-Based Systems	8
3. SYSTEM ANALYSIS	8
3.1 Functional requirements	9
3.2 Non-Functional requirements	10

3.3 Software requirements	11
3.4 Hardware requirements	12
3.5 Risk Assessment & Mitigation	12
3.6 Deployment Strategy	12
3.7 Summary	12
4. SYSTEM DESIGN	13
4.1 Introduction	13
4.2 UML Diagrams	13
4.3 Modules	17
5. IMPLEMENTATION & RESULTS	19
5.1 Introduction	19
5.2 Method of Implementation	19
5.3 Algorithm and Flowcharts	21
5.4 Control Flow of the implementation	23
5.5 Sample Code	29
5.6 Output Screens	33
6. TESTING	36
6.1 Introduction to Testing	36
6.2 Types of Tests considered	37
6.3 Various Test case scenarios considered	38
6.4 Performance Testing	38
6.5 Security Testing	38
6.6 Cross-Browser and Device Compatibility Testing	38

6.7 Regression Testing	39
6.8 Limitations Noted During Testing	39
6.9 Logging and Debugging	39
7. CONCLUSION AND FUTURE ENHANCEMENT	40
7.1 Project Conclusion	40
7.2 Future Enhancement	41
8. REFERENCE	45
8.1 Journals	45
8.2 Books	45
8.3 Sites	45

LIST OF FIGURES

S. No.	Figure No.	Name of the Figure	Page No.
1	4.2.1	Use case diagram	13
2	4.2.2	Class Diagram	14
3	4.2.3	Sequence diagram	15
4	4.2.4	Activity diagram	16
5	5.6.1	Welcome Page	33
6	5.6.2	Toddler data input screen	34
7	5.6.3	Questionnaire	34
8	5.6.4	Prediction results page	35
9	5.6.5	Risk level display with brief recommendations	35

LIST OF TABLES

S. No.	Table No.	Name of the Table	Page No.
1.	6.3	Various Test Case Scenarios Considered	38

LIST OF ABBREVIATIONS

S.No.	Abbreviation	Definitions
1	ASD	Autism Spectrum Disorder
2	M-CHAT	Modified Checklist for Autism in Toddlers
3	CARS	Childhood Autism Rating Scale
4	API	Application Programming Interface

1. INTRODUCTION

Toddler Guardian is a web-based autism screening system developed to assist parents, caregivers, and early childhood professionals in identifying early indicators of Autism Spectrum Disorder (ASD) in toddlers. The application integrates a research-backed behavioral questionnaire with a machine learning-powered prediction model to deliver real-time, accessible, and informative autism risk assessments. Designed with modern technologies and a user-centered approach, the platform aims to promote early detection, increase public awareness, and guide timely professional intervention.

This system is particularly relevant in environments where mental health resources are limited or delayed. By offering a free, convenient, and non-invasive screening experience, Toddler Guardian empowers users to evaluate a toddler's behavioral development and determine whether further clinical evaluation is warranted. Its scalability and ease of deployment make it a potential cornerstone in public health efforts for early ASD detection.

1.1 Motivation

The underlying motivation for developing Toddler Guardian is the critical importance of early intervention in Autism Spectrum Disorder. Studies have consistently shown that earlier detection leads to better cognitive, social, and behavioral outcomes for affected children. However, many children remain undiagnosed until much later, primarily due to lack of awareness, limited healthcare access, or the subtlety of symptoms during early developmental stages.

Parents and caregivers are often the first to observe atypical behaviors, but they may not recognize these as potential signs of ASD or may hesitate to seek professional advice due to social stigma or denial. Moreover, existing diagnostic processes are often lengthy and involve significant waiting times. Toddler Guardian seeks to close this gap by providing an intuitive, low-barrier tool that makes it easier for families to start the evaluation process sooner rather than later. By harnessing machine learning, the tool can reveal hidden patterns in data that may be too nuanced for human observation alone.

1.2 Problem Definition

Autism Spectrum Disorder is a lifelong neurodevelopmental condition characterized by challenges in communication, social interaction, and repetitive behaviors. Early detection is difficult due to the diverse manifestation of symptoms, many of which may be subtle or dismissed as temporary developmental delays. Traditional diagnostic methods require trained professionals, costly assessments, and multiple visits, making early diagnosis inaccessible for many families.

Compounding this issue is a general lack of knowledge about autism in the general population. Many parents might struggle to differentiate between typical toddler behavior and early symptoms of ASD. This leads to delays in seeking help, which can have long-lasting developmental consequences. Toddler Guardian aims to reduce these delays by providing a widely accessible, user-friendly, and fast tool that allows caregivers to complete a preliminary screening in the comfort of their home. The tool helps bridge the gap between initial concern and professional diagnosis, thus fostering earlier support for children who need it.

1.3 Objectives of the Project

The main goal of the Toddler Guardian project is to develop a responsive and educational screening tool that assists in the early identification of ASD symptoms. The platform intends to simplify the initial evaluation process and guide users with actionable insights based on the risk assessment outcome.

The application offers a simplified and accessible screening interface that can be used by individuals with no prior technical or clinical knowledge. It provides immediate, easy-to-understand assessments that categorize autism risk levels while maintaining scientific accuracy. The system encourages timely medical consultation when a high risk is identified and is designed to function across multiple platforms including smartphones, tablets, and desktop computers.

The tool also aims to contribute to public health education by raising awareness of early developmental red flags associated with autism. By doing so, it supports not just individual families, but also the broader mission of early detection and intervention at a community level.

1.4 Limitations of the Project

While Toddler Guardian offers a promising approach to early ASD screening, it is essential to acknowledge its limitations to ensure its proper use and continued development. One of the key limitations is that it is strictly a screening tool, not a diagnostic instrument. It should not replace clinical evaluation and is intended to guide, not confirm, a diagnosis.

The reliability of the tool heavily depends on the accuracy of user input. Misinterpretation of questions or intentional/unintentional misinformation can skew results. Furthermore, the training data used for the machine learning model may lack comprehensive demographic diversity, which could limit the model's generalizability across different ethnic, cultural, or regional populations.

As an online platform, the system requires internet access, which may restrict usability in remote or underprivileged areas. Additionally, since it processes sensitive personal and developmental data, privacy and data security are critical concerns. The system must adhere to regulations such as the General Data Protection Regulation (GDPR) and Health Insurance Portability and Accountability Act (HIPAA) to protect user data and build trust.

1.5 Scope of the Project

The scope of Toddler Guardian includes developing a complete and functioning prototype that encompasses frontend, backend, and predictive analytics. It is intended to be a self-service screening application that can operate independently or as part of a broader healthcare referral system. The system supports data collection, real-time prediction, and user education. Future scope may involve integration with pediatric electronic health records (EHR), expansion into mobile applications, and multilingual support to enhance usability across diverse populations.

1.6 Relevance in Current Healthcare Scenario

With mental health awareness gaining momentum and early childhood intervention emerging as a global priority, a tool like Toddler Guardian aligns well with current healthcare strategies. The COVID-19 pandemic highlighted the importance of telehealth and digital health solutions, making online diagnostic support more relevant than ever. In this context, the tool serves as a timely innovation that complements the healthcare ecosystem by offering a proactive, tech-driven solution for early autism screening.

In summary, Toddler Guardian represents a meaningful application of artificial intelligence and behavioral science in public health. It empowers users, reduces barriers to early intervention, and supports better developmental outcomes through timely guidance and accessible technology.

2. LITERATURE SURVEY

2.1 Introduction

Autism Spectrum Disorder (ASD) is a developmental condition characterized by challenges in social interaction, communication, and repetitive behaviors. Early diagnosis is critical for timely intervention, which has been shown to significantly improve long-term outcomes for children with ASD. Over the past decade, there has been considerable research into both traditional and technology-assisted approaches to ASD diagnosis and screening. This literature survey presents a comparative analysis of existing diagnostic methods and technologies, and explains how the proposed system aims to improve upon them.

2.2 Existing System

Several tools and systems have been developed to screen and diagnose ASD. Most of these rely on structured clinical observations or parent-reported questionnaires. The following are notable existing systems:

Traditional Screening Tools

2.2.1 M-CHAT (Modified Checklist for Autism in Toddlers): A widely used screening tool designed for children aged 16 to 30 months. It consists of a series of yes/no questions completed by the parent, aiming to detect early signs of autism.

2.2.2 ADOS (Autism Diagnostic Observation Schedule): A gold standard diagnostic tool administered by trained clinicians through structured activities and observation.

2.2.3 CARS (Childhood Autism Rating Scale): A behavior rating scale intended to help diagnose autism in children by identifying the severity of symptoms.

Limitations:

2.2.4 Highly dependent on expert interpretation.

2.2.5 Subject to parental bias or misreporting.

2.2.6 Can be time-consuming and not accessible in rural or low-resource settings.

2.2.7 Often applied only after symptoms become noticeable.

Digital and AI-Based Approaches

Recent research has applied machine learning to detect ASD based on various data inputs such as questionnaire responses, speech patterns, facial expressions, and neuroimaging. Some notable developments include:

2.2.8 Machine Learning Classifiers: Algorithms such as Decision Trees, Random Forest, SVM, and Neural Networks have been used on medical and behavioral datasets to classify ASD presence.

2.2.9 Mobile Apps and Web Portals: Some applications collect behavioral data through gamified tests or questionnaires and give preliminary ASD assessments (e.g., Cognoa, AutismAI).

Limitations:

2.2.10 Many systems are closed-source or commercial, lacking transparency.

2.2.11 High reliance on large, balanced datasets.

2.2.12 Some models lack real-world validation or clinical approval.

2.3 Proposed System

The proposed system seeks to address the limitations of both traditional and modern systems by offering an open, accessible, and interpretable platform for ASD screening in toddlers. It leverages machine learning algorithms trained on publicly available datasets (e.g., the ASD Toddler Dataset from UCI Repository) to predict ASD risk based on features such as:

2.3.1 Age, gender, and demographic information.

2.3.2 Behavioral patterns reported by parents.

2.3.3 Developmental milestones and responses to specific questions.

Key Features of the Proposed System:

2.3.4 User-Friendly Interface: Allows parents or clinicians to input data easily through a web form.

2.3.5 Automated Prediction: Provides real-time results indicating low, medium, or high risk of ASD.

2.3.6 Data-Driven: Uses trained ML models to minimize human error and subjective judgment.

2.3.7 Privacy-Aware: Ensures user data is handled securely with confidentiality.

2.3.8 Open to Future Expansion: Can integrate speech/video input, expand to other age groups, and support multi-language interfaces.

Advantages Over Existing Systems:

2.3.9 Reduces dependency on specialist availability.

2.3.10 Offers faster, preliminary screening that can lead to earlier intervention.

2.3.11 Easy to deploy in remote or underserved communities with minimal setup.

2.3.12 Continuously improvable as more data is collected and analyzed.

2.4 Advantages of Traditional Systems

Traditional systems like M-CHAT, ADOS, and CARS have played a significant role in the early detection and diagnosis of Autism Spectrum Disorder. These tools are designed and validated through extensive clinical studies and provide standardized methods of assessment. Because they are typically administered by trained professionals, the diagnostic process benefits from expert judgment and controlled observational settings. This makes them highly reliable in structured clinical environments. Moreover, the long-standing use of these tools ensures familiarity among healthcare providers, making integration into existing medical protocols seamless.

2.5 Limitations of Traditional Systems

Despite their advantages, traditional diagnostic systems exhibit several limitations that hinder widespread and early diagnosis. One of the most pressing issues is their heavy reliance on expert interpretation, which makes them inaccessible in rural or underserved regions where specialists may not be available. Additionally, tools that depend on parent-reported questionnaires are vulnerable to subjective biases or misreporting, reducing the accuracy of assessments. The process can also be time-consuming, often requiring multiple appointments and long wait times for results. Furthermore, these tools are often used only after symptoms become visible, delaying the opportunity for earlier intervention when it is most effective.

2.6 Advantages of AI-Based Methods

AI-based methods offer a transformative approach to autism screening by leveraging computational efficiency and data-driven insights. Machine learning models can be trained on large datasets to recognize patterns that may not be easily visible to clinicians. This allows for faster processing and more objective predictions. The use of digital platforms enables remote accessibility, which is particularly beneficial for communities with limited healthcare infrastructure. Additionally, AI models can be continuously improved over time as more data becomes available, enhancing their accuracy and adaptability. These systems can provide preliminary assessments that prompt earlier clinical evaluations, thus reducing delays in intervention.

2.7 Limitations of AI-Based Methods

Although promising, AI-based autism screening tools are not without their challenges. Many systems are developed as proprietary or commercial solutions, limiting transparency and preventing external validation. The performance of these models often depends on large, well-balanced datasets, which may not always be available, especially for underrepresented populations. Some AI systems lack clinical validation and have not been tested in real-world conditions, raising concerns about their reliability. In addition, the requirement for specialized hardware such as eye trackers or high-resolution cameras may limit practical deployment. Privacy concerns also arise, particularly when sensitive data is collected online, emphasizing the need for secure data handling and ethical model deployment.

3. SYSTEM ANALYSIS

System analysis plays a crucial role in shaping the development process by identifying what the system should do (functional requirements), how it should perform (non-functional requirements), and the tools and platforms required to support it. The Autism Screening System is a data-driven web application built using machine learning, and its analysis ensures reliability, efficiency, and user-centric design.

3.1 Functional Requirements

The functional requirements describe the core operations and behaviors that the system must be able to perform to achieve its purpose.

3.1.1 Welcome Interface: The system begins with a welcoming interface that serves as the landing page. This page introduces the purpose of the platform and offers guidance to new users. The design is friendly and accessible, providing simple instructions, a brief overview of the application's goals, and options to start the screening process.

3.1.2 Basic Information Input Form: Once the user initiates the screening, the system presents a structured form to collect basic demographic and health-related information about the toddler. These include the child's age, gender, family history of ASD, presence of jaundice, ethnicity, and parental relation. The form uses dropdowns, radio buttons, and input boxes to maintain clarity and minimize errors.

3.1.3 Questionnaire Module: After the basic information is submitted, users are guided through the Q-Chat-10 questionnaire. Each question is presented one at a time, in sequence, to maintain the user's focus and improve answer accuracy. This also ensures that the experience is not overwhelming for parents or guardians unfamiliar with medical screening processes.

3.1.4 Input Validation: All user inputs undergo strict validation checks. Mandatory fields must be filled, and invalid or inconsistent values are flagged in real-time. For instance, age must fall within an appropriate range (e.g., 12–48 months), and textual entries like gender must match predefined options. These validations are critical to prevent erroneous data from being passed to the machine learning model.

3.1.5 Risk Prediction Engine: Upon form completion, the data is preprocessed and fed into a trained XGBoost classifier model. This engine has been trained on real-world datasets of toddlers screened for ASD and can detect complex patterns to predict whether the child is at Low, Medium, or High risk for ASD. The model combines both behavioral and demographic features to deliver a robust assessment.

3.1.6 Result Summary Display: After prediction, a result summary is generated and displayed to the user. This summary includes the overall risk category, the child's total score on the Q-Chat questionnaire, and specific questions that raised concern. The results are presented in a simple, interpretable format to ensure clarity even for non-technical users.

3.1.7 Recommendation Generator: Based on the predicted risk level, the system offers personalized recommendations. These include suggestions for further professional evaluation, early intervention programs, and online resources. The goal is not just to inform but to guide users toward timely and actionable steps.

3.1.8 Admin and Feedback Module (Optional for Enhancement): To support future improvements, an admin portal may be introduced to collect anonymous usage statistics, user feedback, and error logs. This backend system would help monitor engagement levels and ensure the ongoing relevance and reliability of the application.

3.2 Non-Functional Requirements

These requirements define how the system performs its tasks and how users interact with it.

3.2.1 Usability: The application is designed to be intuitive and user-friendly. Special care is taken to make it accessible for individuals with little to no technical background. Large buttons, clear fonts, step-by-step instructions, and tooltips help enhance ease of use.

3.2.2 Responsiveness: All user actions, such as navigation between questions or receiving prediction results, occur within a few seconds. The system is optimized for fast loading and minimal delays, ensuring a smooth user experience even on slower connections.

3.2.3 Security: Security is paramount, especially because the application handles sensitive information. While no personally identifiable data is stored unless explicitly enabled, all interactions are encrypted. The system ensures that session data is cleared after each use to prevent leaks.

3.2.4 Scalability: The modular architecture allows for easy scaling. New questions, languages, risk models, or even entire modules (such as chatbot support or advanced analytics) can be integrated without major restructuring of the codebase.

3.2.5 Reliability & Availability: The system should be reliably available during operational hours. Downtime must be minimized using regular server monitoring and error handling mechanisms. In future deployments, cloud-based hosting can be used to ensure 99.9% uptime.

3.2.6 Cross-Platform Compatibility: The application is web-based and tested to function seamlessly across common browsers like Chrome, Firefox, and Edge. It is also responsive and adapts to various screen sizes, making it accessible on smartphones, tablets, and desktops.

3.2.7 Maintainability: The application follows modular programming practices. Each module (data preprocessing, ML model, web interface) is developed and documented separately. This design facilitates debugging and future enhancements.

3.2.8 Accessibility Compliance: As an added advantage, the system ensures compliance with basic accessibility standards (WCAG 2.1). Support for screen readers, alternative text for images, and keyboard navigation are included to serve users with disabilities.

3.3 Software Requirements

The system is built with widely used and reliable tools to support machine learning and web deployment.

- **Operating System:** Windows 7
- **Web Browser:** Google Chrome
- **Software Development Tools:** Visual Studio Code, Jupyter Notebook
- **Programming Language:** Python 3.7
- **Web Framework:** Flask
- **Libraries/Packages:** NumPy, Pandas, Scikit-learn, XGBoost, Joblib, Jinja2
- **ML Toolkit:** scikit-learn and XGBoost
- **Development Environment:** Flask Local Server with optional virtual environments for package management

3.4 Hardware Requirements

To run the application efficiently on a local machine or development environment, the following minimum hardware configuration is recommended:

- **Processor:** Intel Core i3
- **RAM:** 4 GB minimum
- **Hard Disk:** 40 GB
- **Monitor:** Standard display (resolution of 1024x768)
- **Internet Connection:** Required for accessing web modules and downloading dependencies

3.5 Risk Assessment & Mitigation

Before deployment, potential technical and operational risks are identified and mitigated. These include issues such as data bias in the training set, model overfitting, user misinterpretation of results, and network failures. Techniques such as data augmentation, cross-validation, logging, and fallback instructions are used to handle these scenarios.

3.6 Deployment Strategy

Initially, the system is deployed locally for testing and refinement. Once validated, it can be migrated to a production environment via platforms such as Heroku, AWS, or Azure. Continuous Integration/Continuous Deployment (CI/CD) pipelines may be introduced for future updates and bug fixes.

3.7 Summary

The detailed system analysis ensures that the application is designed with a focus on user experience, data integrity, performance, and future scalability. With well-defined functional and non-functional requirements, the Autism Screening Tool is structured to be both practical and impactful, ready for real-world use and future development.

4. SYSTEM DESIGN

4.1 Introduction

System design specifies the architecture, components, interfaces, and data flow. This ensures that the functional and non-functional requirements are implemented efficiently and effectively.

4.2 UML Diagrams

4.2.1 Use Case Diagram

This use case diagram illustrates a child health screening system that assists users in evaluating possible treatment options using a machine learning (ML) model. The process begins when the user initiates the screening and enters relevant child information. These steps are essential and included as part of the screening workflow. After submitting the data, the system uses an ML model to analyze the information and generate health-related insights. The user can then view treatment options based on the model's output. Additionally, the system allows the user to return to the results or restart the screening process if needed. This diagram effectively captures the interaction between the user and the system components, emphasizing the integration of ML for data-driven decision support.

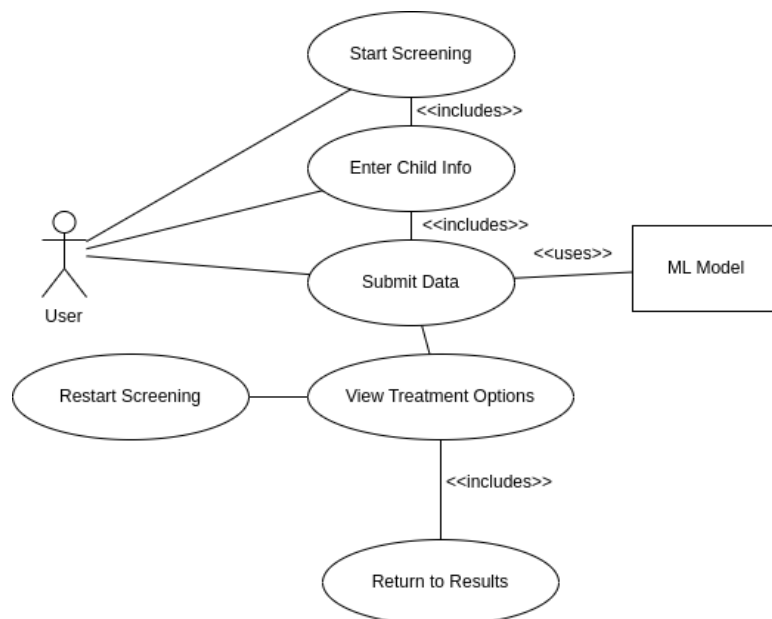


Fig 4.2.1: Use Case Diagram

4.2.2 Class Diagram

This class diagram illustrates the structure and relationships between different components of a child health screening system. The central class **Screening** handles user interactions, offering methods like `startScreening()`, `enterChildInfo()`, and `answerQuestions()` to collect and manage user input. It collaborates with the **MLModel** class, which provides the `predictRisk()` method to analyze user-provided answers and return a risk level. The resulting risk level is stored in the **Result** class as a private attribute `riskLevel`, which can be accessed through the `getRiskLevel()` method. Finally, the **TreatmentOptions** class allows users to explore possible actions through `viewTreatmentOptions()` and return to view their risk results using `returnToResults()`. This diagram effectively captures the system's architecture and how the components interact through method calls and data sharing.

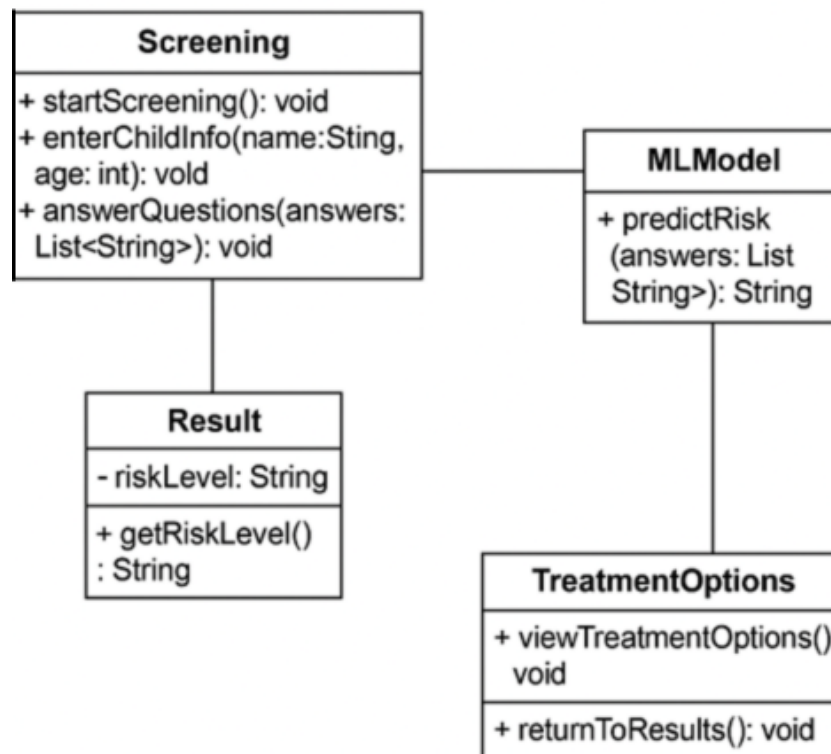


Fig 4.2.2: Class Diagram

4.2.3 Sequence Diagram:

This is a **sequence diagram** that represents the step-by-step interaction between a user and a child health screening system, including its components like the ML model, result viewer, and treatment options. The process begins when the user initiates the screening by calling `startScreening()`. The system then collects the child's basic information such as name and age via `enterChildInfo()`, followed by answering diagnostic questions through `answerQuestions()`. These answers are sent to the integrated **ML Model** using the `predictRisk()` function, which processes the data and returns a risk prediction via `getRiskLevel()`. Based on this risk level, the user can request to `viewTreatmentOptions()`, and optionally `returnToResults()` to recheck the ML model's output. The final `result()` is then displayed to the user. This diagram effectively captures the dynamic flow of control and data between components in the system.

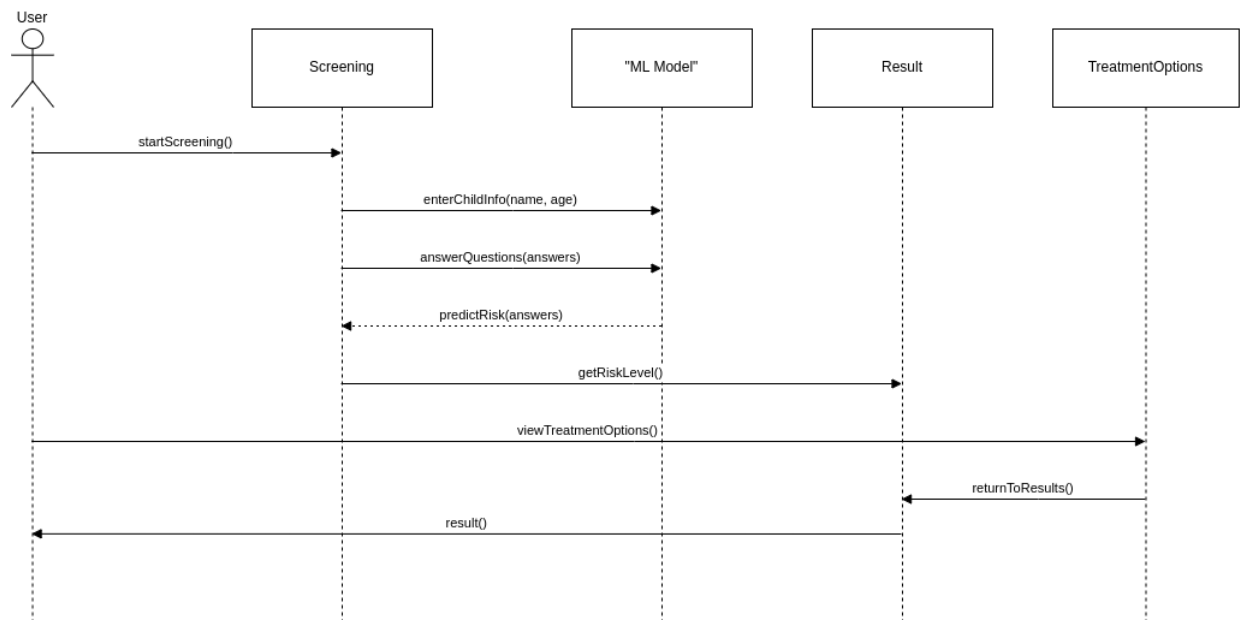


Fig 4.2.3: Sequence Diagram

4.2.4 Activity Diagram:

This activity diagram outlines the workflow of an autism screening application for toddlers, integrating key modules such as Authentication, Data Collection, Prediction, Results, and Reporting. The user begins by entering child details, which are processed and sent to a machine learning model that predicts the risk level (Low/Medium/High). Based on the result, authenticated users can view treatment options and download reports. The flow also includes checks for authentication, options to restart screening, and ensures a smooth transition between modules for a complete user experience.

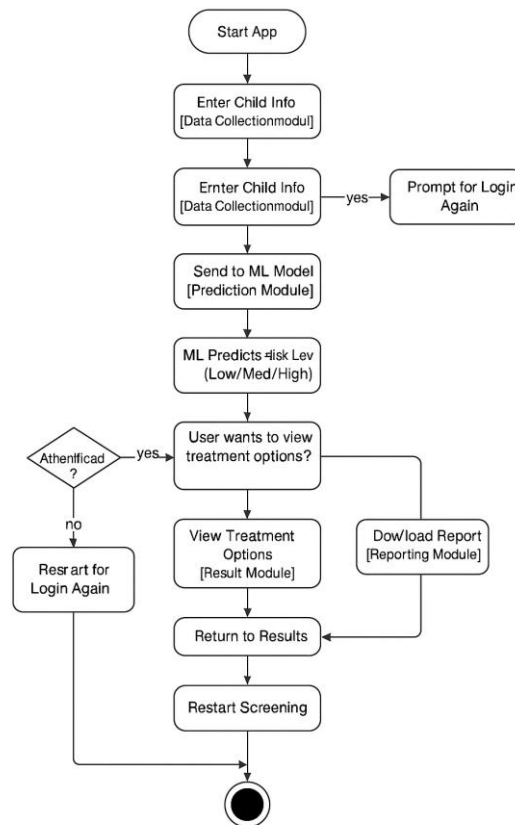


Fig 4.2.4: Activity Diagram

4.3 Modules

The system is composed of several modular components, each responsible for a distinct stage in the autism risk screening pipeline. The following outlines the key modules of the application:

Authentication Module

This module handles the security and user management of the system. It enables users to:

- 4.3.1 Register new accounts using credentials such as email and password.
- 4.3.2 Log in to existing accounts securely.
- 4.3.3 Manage active sessions for continuous and protected use of the application.
- 4.3.4 Prevent unauthorized access to sensitive data like screening results.

Data Collection Module

This module provides a user-friendly interface to gather essential input from users. It includes:

- 4.3.5 Forms to input child-specific data (e.g., name, age, gender).
- 4.3.6 A standardized questionnaire (e.g., 10 ASD-related behavioral questions).
- 4.3.7 Basic input validation to ensure data completeness and correctness.

Preprocessing Module

Before passing the user inputs to the ML model, this module transforms the raw data into a format suitable for prediction. Tasks performed include:

- 4.3.8 Encoding categorical responses into numerical values.
- 4.3.9 Normalizing or scaling numerical data if required.
- 4.3.10 Handling missing or inconsistent inputs gracefully.
- 4.3.11 Structuring the processed data as a feature vector for model inference.

Prediction Module

The core of the system, this module uses a pre-trained machine learning model to predict ASD risk. Its functions include:

- 4.3.12 Receiving processed data from the preprocessing module.
- 4.3.13 Executing the ML model to determine the risk score.
- 4.3.14 Classifying the score into categories: **Low**, **Medium**, or **High** risk of ASD.

Result Module

Once a prediction is made, this module presents the output in an understandable manner. It:

- 4.3.15 Displays the risk level (e.g., "Low Risk").
- 4.3.16 Provides a brief interpretation to assist non-technical users in understanding the result.
- 4.3.17 Suggests whether professional diagnosis is recommended based on the score.

Reporting Module

This module supports the generation and dissemination of results in a shareable format. Key features include:

- 4.3.18 Creating downloadable PDF or summary reports for clinicians or parents.
- 4.3.19 Including date/time, input data, and predicted risk.
- 4.3.20 Providing basic guidance on next steps or referrals.

5. IMPLEMENTATION & RESULTS

5.1 Introduction

The implementation phase is where the proposed autism screening system is brought to life. This system is designed to be simple, interactive, and accessible, allowing users to assess the likelihood of Autism Spectrum Disorder (ASD) in toddlers. By combining a user-friendly interface with a powerful machine learning backend, we aim to provide a reliable and supportive tool for early detection. The implementation focuses on seamless integration of the frontend, backend, and the XGBoost model, ensuring that users receive accurate predictions along with appropriate guidance. Through this process, we aim to contribute toward improving early intervention and awareness in ASD cases.

5.2 Method of Implementation

The implementation of the autism screening system follows a structured and modular development approach, ensuring that each component — frontend, backend, and machine learning model — is developed, tested, and integrated effectively. This division of concerns allows for easy debugging, code maintenance, and clear understanding of the responsibilities of each module.

5.2.1 Development Begins with the Frontend (User Interface)

The project development started with the design and implementation of the user interface using HTML templates integrated into a Flask framework. The frontend plays a crucial role in guiding users through the screening process in a clear, friendly, and structured manner. The interface includes a basic information form for collecting user details such as age, gender, and family history, followed by a series of 10 behavior-related questions inspired by the Q-Chat-10 questionnaire. Each question is displayed on a separate page to keep the user focused and avoid overwhelming them. User responses are collected through simple radio buttons (Yes/No), making the system accessible even for users without technical knowledge. Emphasis was placed on keeping the interface intuitive and mobile-friendly.

5.2.2 Backend Handles Data Validation and Processing

Once the frontend was established, the backend was implemented using Python and the Flask web framework. It serves as the bridge between user input and model prediction. The backend performs key tasks including input validation, response transformation, and session tracking. Collected responses are validated and mapped to the required input format using a rule-based scoring logic. For example, some questions interpret a “No” as a potential risk, while others interpret “Yes” as a risk. This logic is implemented in the backend using dictionaries and conditional statements. Furthermore, the backend ensures smooth navigation across question pages and processes the final response set to prepare it for model input.

5.2.3 Machine Learning Model Trained Using Labeled ASD Dataset

The predictive engine at the core of this system is a machine learning model trained using a publicly available labeled dataset for Autism Spectrum Disorder (ASD) screening. The dataset was first cleaned and preprocessed by removing unnecessary columns and encoding categorical variables using `LabelEncoder`. A `StandardScaler` was used to normalize the feature values, ensuring consistency and better performance during model training. The model used is an **XGBoost Classifier**, chosen for its robustness and ability to handle tabular data with categorical variables. To ensure fair evaluation, **Stratified K-Fold Cross-Validation** was implemented. The model was trained to classify input data as either indicating ASD traits or not, and then serialized using `joblib` for use in the web application.

5.2.4 Final System Integrates Frontend, Backend, and ML Model

After independently developing and testing the user interface, backend logic, and machine learning model, all components were integrated into a single Flask-based web application. The frontend sends user responses to the backend, which in turn preprocesses the input, applies the trained model, and analyzes risk based on the user's answers. The final result — whether or not ASD traits are predicted — along with risk level and a summary of concern areas, is rendered back to the user through the results page. The successful integration of all these components has resulted in an end-to-end functional system capable of providing early screening support for autism in toddlers based on user input.

5.3 Algorithm and Flowcharts

5.3.1 Algorithm Used: XGBoost Classifier

For this autism detection project, the **XGBoost Classifier (Extreme Gradient Boosting)** was selected as the core machine learning algorithm due to its high performance, efficiency, and robustness on structured/tabular data. XGBoost is an advanced implementation of gradient boosting decision trees, specifically optimized for speed and accuracy. It builds an ensemble of decision trees in a sequential manner, where each new tree attempts to correct the errors made by the previous ones. This approach allows the model to focus on difficult-to-predict instances and improve prediction accuracy iteratively.

XGBoost uses a technique called **gradient boosting**, which involves minimizing a loss function (in this case, log loss for classification) by adding new models that predict the residuals (errors) of previous models. The algorithm incorporates regularization parameters (λ , α , and γ) to control model complexity and prevent overfitting, making it suitable for real-world datasets that may have noise or imbalance. Additionally, features such as **column subsampling**, **row subsampling**, and **early stopping** provide further control over training, improving generalization and computational efficiency.

In the context of autism prediction, XGBoost was trained on a dataset of toddler screening responses, demographic information, and family history of autism. The input features were preprocessed using standard scaling and label encoding, ensuring uniform data representation. The classifier was trained with hyperparameters like `n_estimators=100`, `learning_rate=0.03`, `max_depth=3`, and `gamma=0.5`, which were selected after experimentation for optimal balance between bias and variance. The model achieved high accuracy and generalization during cross-validation, making it a strong choice for this healthcare-related classification task.

Moreover, XGBoost's compatibility with Python and its ability to integrate seamlessly with Scikit-learn pipelines made it easy to implement, evaluate, and deploy. The final model was saved using the joblib library for real-time prediction on new user input. Its fast training time, built-in handling of missing values, and support for feature importance ranking further enhanced the interpretability and practicality of the system. Therefore, the XGBoost Classifier served as a reliable and effective algorithm for detecting ASD risk levels in toddlers.

5.3.2 Flowchart Steps

The system follows a structured sequence of steps that ensure smooth interaction from data input to final prediction output. These steps represent the logical flow of how the autism screening application functions internally. Each step plays a vital role in ensuring that the model receives data in the correct format and can make an accurate prediction based on the input.

5.3.2.1 User Inputs Toddler Data

The process begins when the user, typically a parent or guardian, visits the application and enters relevant information about the toddler. This includes demographic details such as age in months, sex, ethnicity, whether the child has experienced jaundice, family history of ASD, and the relationship of the respondent to the child. Following this, the user is guided through a series of behavior-related questions (e.g., eye contact, pretend play, language development), answering each with a simple Yes or No. These responses serve as key indicators for the system to evaluate developmental traits associated with ASD.

5.3.2.2 Data is Preprocessed (Normalized, Encoded)

Once all inputs are collected, the backend system preprocesses the data before feeding it into the machine learning model. This step involves encoding categorical variables using label encoding to convert textual inputs like "Male", "Yes", or "Mother" into numerical values. Numerical features are then standardized using a scaler to ensure that all features are on the same scale, improving model performance. Additionally, logical transformation is applied to question responses — for example, certain questions where a "No" response indicates higher risk are inverted accordingly. This preprocessing ensures that the model receives clean, uniform, and meaningful input.

5.3.2.3 Data is Fed into ML Model

After preprocessing, the data is organized into a feature array that matches the structure expected by the trained machine learning model. This feature vector is then passed to the model — specifically, an XGBoost Classifier — which processes the input using patterns it learned during training on the ASD dataset. The model has been trained to recognize combinations of features that are strongly associated with the presence or absence of ASD traits, allowing it to make an informed prediction.

5.3.2.4 Model Returns ASD Prediction

Once the input is processed, the model outputs a prediction — indicating whether or not the toddler shows signs of Autism Spectrum Disorder. This output is in numerical form and is converted back into a human-readable label using inverse transformation. The system does not stop at a binary prediction; it also calculates a Q-Chat-10 based score and identifies which questions contributed to the potential risk. This makes the prediction more interpretable to the user.

5.3.2.5 Risk Level is Displayed to the User

The final step involves displaying the results to the user in a clear and supportive manner. The results page includes the ASD prediction, the calculated risk level (Low, Medium, or High), and a list of specific questions that indicated concern. This helps the user understand not just the final result, but also the reasoning behind it. The overall goal is to provide early awareness and encourage parents or guardians to seek professional guidance if high risk is detected. The interface ensures that the user leaves the session with meaningful insights and clarity regarding their child's developmental health.

5.4 Control Flow of the Implementation

This section describes the sequential steps followed during the implementation of the Autism Detection System, including dependency setup, data preprocessing, model training, and prediction logic.

5.4.1 Importing Dependencies

In any machine learning-based application, especially one dealing with healthcare-related predictions, it is essential to begin with importing the right set of libraries. These dependencies simplify complex processes, enable efficient data handling, and support building scalable applications. In our autism screening system, several Python packages were utilized to enable functionalities such as reading and cleaning the dataset, encoding categorical variables, building and training the machine learning model, creating visualizations, and deploying the application through a web interface. The following subsections provide a detailed explanation of the purpose and role of each imported module.

5.4.1.1 Pandas and NumPy

The foundational data-handling libraries used were `pandas` and `numpy`. Pandas provided robust support for reading structured datasets like CSV files, handling missing values, filtering and selecting relevant columns, and transforming the data into formats suitable for modeling. It allowed us to treat the dataset as a `DataFrame`, simplifying exploratory analysis and preprocessing. NumPy, on the other hand, was extensively used for numerical operations and manipulating array-based data. Before passing the user inputs to the model, NumPy was used to structure the input into arrays that could be standardized and reshaped for prediction. Its integration with machine learning models and mathematical efficiency made it essential for this system.

5.4.1.2 Joblib

Persisting machine learning models and preprocessing objects is important to ensure reusability and scalability. The `joblib` library was used to serialize and deserialize the trained XGBoost model, the fitted `StandardScaler`, and the `LabelEncoder` mappings. These saved objects were loaded during the deployment phase through the Flask application, avoiding the need to retrain the model every time a prediction was requested. This not only reduced computation time but also made the system suitable for real-time deployment. Joblib is known for its high performance on large NumPy arrays and is more efficient than the built-in `pickle` library for such use cases.

5.4.1.3 Matplotlib and Seaborn

Visualization is a critical part of data science and model evaluation. Libraries such as `matplotlib.pyplot` and `seaborn` were used to visualize the relationships between features and classes, as well as to analyze correlations in the data. These visualizations provided insights during the development phase, helping in identifying outliers, imbalanced classes, or feature redundancy. Heatmaps, bar plots, and distribution plots generated by these libraries allowed us to interpret the data better and make decisions like which columns to drop or encode. Though not a part of the deployed system, their role in the analytical phase was crucial in shaping the final model.

5.4.1.4 Scikit-learn Modules

The `scikit-learn` (or `sklearn`) library played a central role in both preprocessing and evaluation. For data splitting, `StratifiedKFold` ensured that the training and testing sets maintained the original class proportions, which is vital in classification problems involving medical predictions like autism detection. `LabelEncoder` converted categorical text-based columns such as gender, ethnicity, and respondent type into numerical labels, making them compatible with the model. `StandardScaler` standardized the input features by removing the mean and scaling to unit variance, which helped the model perform more consistently. Additionally, evaluation metrics like `accuracy_score` and `classification_report` were used to measure how well the model performed on unseen data, giving details like precision, recall, and F1-score.

5.4.1.5 XGBoost Classifier

To build a robust and efficient predictive model, the `XGBClassifier` from the XGBoost library was employed. XGBoost is an advanced gradient boosting algorithm that performs well with structured/tabular data and handles missing values efficiently. It also allows for fine control over model parameters such as learning rate, tree depth, and regularization. This classifier provided high accuracy and handled the relatively small dataset effectively. Its built-in handling of overfitting, support for parallel processing, and flexible API made it a strong candidate for use in our health screening model. The model was tuned with appropriate hyperparameters like `n_estimators`, `learning_rate`, and `max_depth` to ensure reliable predictions.

5.4.1.6 Flask for Web Interface

To make the system user-friendly and accessible, we deployed the prediction model using the `Flask` web framework. Flask is a lightweight and flexible Python-based microframework that supports routing, template rendering, and form handling. It enabled the creation of a complete web interface where users could enter their responses to the autism screening questions, which were then passed to the backend for prediction. The Flask server processed inputs, performed necessary preprocessing using the previously saved encoders and scalers, and then passed the input to the model to receive a prediction. The result was then rendered back to the user using HTML templates, making the process seamless and interactive.

5.4.1.7 Additional Flask Utilities

Several additional utilities were imported from Flask to enhance the interaction between the user and the server. These included `request` to fetch form data submitted by users, `redirect` and `url_for` for navigation between pages, and `jsonify` to return structured JSON responses in case of API-based usage or errors. These functions allowed the application to handle user sessions, store temporary responses, and route users through a multi-step questionnaire. Collectively, these utilities made the application dynamic and responsive to user actions, thereby enabling a smooth and logical flow through the prediction process.

5.4.1.8 Stratified K-Fold Cross-Validation

To evaluate the model fairly and avoid biased results due to class imbalance, we implemented **Stratified K-Fold Cross-Validation** using `StratifiedKFold` from `sklearn.model_selection`. In medical or psychological datasets, such as the autism screening dataset, class imbalance is common — where the number of positive cases (with ASD traits) is much lower than negative cases. Stratified K-Fold ensures that each fold maintains the same distribution of target classes as the original dataset. This leads to more reliable evaluation metrics across each fold. The model is trained and validated multiple times on different subsets, and the average performance gives a more generalized understanding of how the model will perform in real-world scenarios.

5.4.1.9 JSON Handling and Session Storage

Since the application spans multiple screens (basic details form, followed by 10 individual questions), intermediate user responses had to be preserved across routes. This was done by storing responses in a shared dictionary (details) within the server session. To safely exchange data (such as errors or prediction results) between the backend and frontend, JSON formatting using `jsonify()` was employed. JSON is lightweight, human-readable, and compatible with both frontend and backend processing. Though not a full session-based system, this minimal state-holding approach simplified user flow without requiring external storage like a database.

5.4.1.10 Model Interpretability (Risk Analysis Function)

To provide meaningful feedback beyond just “Yes” or “No” predictions, a custom `analyze_risk()` function was implemented. This function evaluated the total number of risk-indicating answers (based on Q-Chat-10 scoring) and determined whether the individual is at low, medium, or high risk. It also listed which specific questions raised red flags, making the results more actionable for caregivers or health professionals. Such interpretability is important for health applications, as users may require explanations to trust and act upon the results. Integrating model prediction with rule-based risk analysis made the solution more transparent.

5.4.1.11 Input Validation and Default Handling

A crucial but often overlooked aspect in real-world applications is handling missing or incorrect input. In this project, certain defaults were provided when users left fields empty or submitted unexpected values. For example, if a categorical input was not recognized by the label encoder (perhaps due to a typo or case mismatch), it was encoded as a neutral or fallback value (e.g., 0). For numerical fields like age, missing values were imputed using average dataset values. This reduced the chances of runtime errors and allowed the system to continue functioning smoothly even with incomplete or noisy input.

5.4.1.12 Q-Chat Scoring Logic

The screening questionnaire in this project is inspired by the **Q-Chat-10**, a simplified version of the Checklist for Autism in Toddlers. Questions are interpreted based on established autism risk patterns. For most questions, a response of “No” (indicating absence of typical behavior) is considered risky, while for a few, a “Yes” is considered risky. This asymmetry was carefully implemented in the `transform_input()` function, ensuring each question's scoring logic matched clinical guidelines. This thoughtful mapping of answers to binary features (0 for safe, 1 for risky) ensured that the scoring was meaningful and clinically aligned.

5.4.2 Data Preprocessing and Model Training

The implementation begins with loading the autism dataset and performing data cleaning operations, such as removing unnecessary columns like unique IDs or pre-calculated scores that could lead to data leakage. Categorical features such as gender, ethnicity, and test completion information are encoded numerically using label encoding, allowing the machine learning model to interpret them. The dataset is then split into input features (X) and target labels (y), followed by the application of **Stratified K-Fold Cross Validation** to maintain class distribution during training and testing. Standardization is performed on numerical data using `StandardScaler`, ensuring that all features contribute equally to the model. The core classifier used is **XGBoost**, selected for its superior performance in tabular classification problems. It is trained on the standardized training set and evaluated using metrics like accuracy and classification reports.

5.4.3 Prediction and Risk Assessment Logic

Once trained, the model is saved using `joblib` for future use without re-training. To make predictions for new users, the system accepts input in dictionary format, matching the structure of the training features. Missing values are handled gracefully with mean imputation or default values. The input is preprocessed in the same way as the training data, and then passed to the XGBoost model for prediction. Alongside this, a risk analysis module evaluates the user's answers to the 10 core screening questions (Q-Chat-10) and computes a risk score. If the total risk score exceeds a defined threshold, the user is flagged as high-risk. This dual system—combining machine learning prediction with rule-based scoring—enhances the model's interpretability and provides a comprehensive output that includes the predicted label, at-risk questions, total score, and overall risk level. This result can be integrated with a web or voice-enabled interface using Django, gTTS, or SpeechRecognition for user-friendly interaction.

5.5 Sample Code

5.5.1 Python: Python is a powerful and easy-to-learn high-level programming language widely used in data science and web development. It is the core language used for both machine learning and backend development in this project.

5.5.2 Flask: Flask is a lightweight and flexible Python web framework used to create the backend

of this system. It allows seamless integration of machine learning models into web applications and enables routing, template rendering, and form handling.

5.5.3 XGBoost: XGBoost (Extreme Gradient Boosting) is a high-performance machine learning algorithm known for its speed and accuracy. It was used in this project to build the ASD (Autism Spectrum Disorder) prediction model based on labeled toddler behavior data.

5.5.4 Joblib: Joblib is a library used to save and load machine learning models efficiently. In this project, it is used to serialize the trained XGBoost model, data scaler, and label encoders for later use in prediction.

5.5.5 Scikit-learn: This library is used for data preprocessing, model evaluation, and utilities like LabelEncoder, StandardScaler, and train_test_split. It complements the XGBoost model by preparing the data before training and prediction.

5.5.6 NumPy: NumPy is used for numerical computations, particularly in handling input arrays for feeding the data into the model during inference.

5.5.7 Pandas: Pandas is used in the model training phase for loading, cleaning, and transforming structured data from the dataset.

5.5.8 HTML & Jinja2: The frontend of the application is built using HTML templates rendered via Flask's Jinja2 engine. It provides a simple and interactive user interface for survey input and result display.

5.5.9 Bootstrap (Optional if used): For styling and responsive design, Bootstrap CSS can be used within HTML templates to improve the visual layout of the form and results page.

5.5.1 XGBoost Model Training Code

5.5.1.1 Data Preprocessing

```
data = pd.read_csv("AutismDataset.csv")
data.drop(columns=["Case_No", "Qchat-10-Score"], inplace=True)

# Encode categorical columns
for col in data.select_dtypes(include=['object']):
```



```
data[col] = LabelEncoder().fit_transform(data[col])
```

5.5.1.2 Feature Selection and Splitting

```
X = data.drop(columns=["Class/ASD Traits "])  
y = data["Class/ASD Traits "]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y)
```

5.5.2.3 Model Training

```
model = XGBClassifier()  
model.fit(X_train, y_train)
```

5.5.1.4 Model Evaluation

```
y_pred = model.predict(X_test)  
print(accuracy_score(y_test, y_pred))
```

5.5.5 Save Model

```
joblib.dump(model, "autism_model.pkl")
```

5.5.6 Risk Analysis (Q-Chat)

```
def analyze_risk(data):  
    score = sum(data[f"A {i}"] for i in range(1, 11))  
    return "High" if score > 3 else "Low"
```

5.5.2 Flask Backend Integration Code

5.5.2.1 Import Libraries and Load Model

```
from flask import Flask, render_template, request, redirect, url_for, jsonify  
import joblib, numpy as np
```

```
# Load model and preprocessing tools  
model = joblib.load("autism_model.pkl")  
scaler = joblib.load("scaler.pkl")  
label_encoders = joblib.load("label_encoders.pkl")
```

5.5.2.2 Initialize Flask App

```
app = Flask(__name__)
```

```
details = {} # Temporary storage for user responses
```

5.5.2.3 Define Autism Survey Questions

```
questions = [  
    {"text": "Does your child look at you when you call his/her name?"},  
    {"text": "How easy is it for you to get eye contact with your child?"},  
    ...  
]
```

5.5.2.4 Home Page Route

```
@app.route('/')  
def index():  
    return render_template('index.html')
```

5.5.2.5 Basic Info Form Route

```
@app.route('/survey', methods=['GET', 'POST'])  
def survey():  
    if request.method == 'POST':  
        details.update({...}) # Collect age, sex, ethnicity, etc.  
        return redirect(url_for('questionnaire', q_num=0))  
    return render_template('basic_form.html')
```

5.5.2.6 Questionnaire Navigation

```
@app.route('/questionnaire/<int:q_num>', methods=['GET', 'POST'])  
def questionnaire(q_num):  
    if request.method == 'POST':  
        answer = request.form.get('answer')  
        details[f'Q{q_num + 1}'] = {"question": ..., "answer": answer}  
        if q_num + 1 < len(questions):  
            return redirect(url_for('questionnaire', q_num=q_num + 1))  
        return redirect(url_for('predict'))  
    return render_template('questions.html', question=..., q_num=q_num)
```

5.5.2.7 Input Transformation

```
def transform_input(user_input):  
    transformed = {  
        "Age_Mons": int(user_input["age"]),  
        ...  
    }  
  
    # Map answers to A1–A10 based on concern level  
    for i in [1,2,...]:
```

```

        transformed[f'A{i}"] = 0 if user_input[f'Q{i}"]["answer"] == "Yes" else 1
    return transformed

```

5.5.2.8 Risk Analysis Function

```

def analyze_risk(user_input):
    score = sum(user_input.get(f'A{i}', 0) for i in range(1, 11))
    if score <= 3:
        risk_level = "Low"
    elif score <= 7:
        risk_level = "Medium"
    else:
        risk_level = "High"
    return ..., score, risk_level

```

5.5.2.9 Preprocess User Input

```

def preprocess_input(user_input):
    user_input = transform_input(user_input)
    for col in ['Sex', 'Ethnicity', ...]:
        user_input[col] = label_encoders[col].transform([user_input[col]])[0]
    return user_input

```

5.5.2.10 Autism Prediction Function

```

def predict_autism(user_input):

    user_input = preprocess_input(user_input)
    input_data = np.array([user_input[col] for col in expected_features]).reshape(1, -1)
    input_data = scaler.transform(input_data)
    prediction = model.predict(input_data)
    ...
    return {
        "prediction": ..., # ASD or Not
        "score": ...,
        "risk_level": ...
    }

```

5.5.2.11 Results Route

```

@app.route('/predict', methods=['POST','GET'])
def predict():
    data = details
    result = predict_autism(data)
    details.clear()
    return render_template("results.html", result=result, questions=questions)

```

5.5.2.12 Run the Flask App

```

if __name__ == "__main__":
    app.run(debug=True)

```

5.6 Output Screens

Screens include

Welcome Page

This platform is designed to assist parents, caregivers, and professionals in identifying early signs of Autism Spectrum Disorder (ASD) in toddlers. Through a simple and interactive questionnaire, users can assess behavioral patterns and receive a predictive analysis powered by machine learning.

Clicking **Start Survey** will begin screening process.

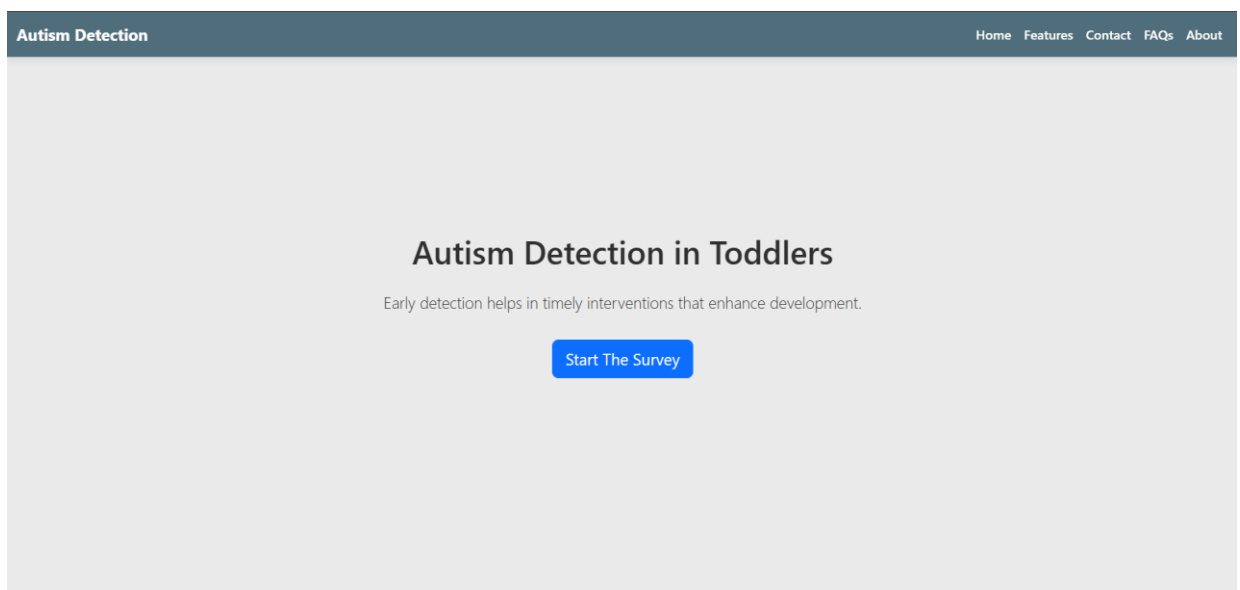


Fig 5.6.1: Welcome Page

Toddler data input screen

The Toddler Data Input Screen is the initial step in the screening process. It collects essential background information about the child that plays a vital role in the autism prediction model. This screen ensures that all relevant demographic and medical details are captured before moving on to behavioral assessment.

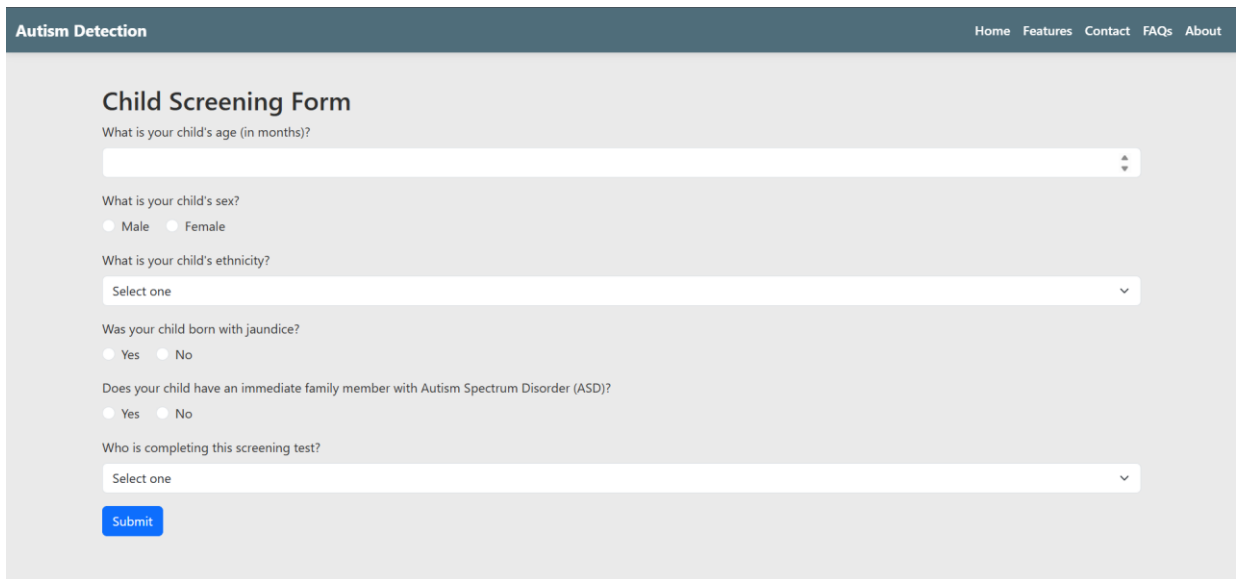
The screenshot shows a web application titled "Autism Detection" in the top left corner. The top right corner contains navigation links: "Home", "Features", "Contact", "FAQs", and "About". The main content area is titled "Child Screening Form". It contains several input fields: a text box for "What is your child's age (in months)?", a radio button group for "What is your child's sex?" with options "Male" and "Female", a dropdown menu for "What is your child's ethnicity?" with the text "Select one", a radio button group for "Was your child born with jaundice?" with options "Yes" and "No", a radio button group for "Does your child have an immediate family member with Autism Spectrum Disorder (ASD)?" with options "Yes" and "No", and another dropdown menu for "Who is completing this screening test?" with the text "Select one". At the bottom left of the form is a blue "Submit" button.

Fig 5.6.2: Toddler data input screen

Questionnaire

After entering the basic toddler information, the system guides users through a set of 10 carefully designed questions, asked one at a time. These questions focus on key behavioral indicators related to Autism Spectrum Disorder (ASD), based on the well-established Q-CHAT-10 screening tool.

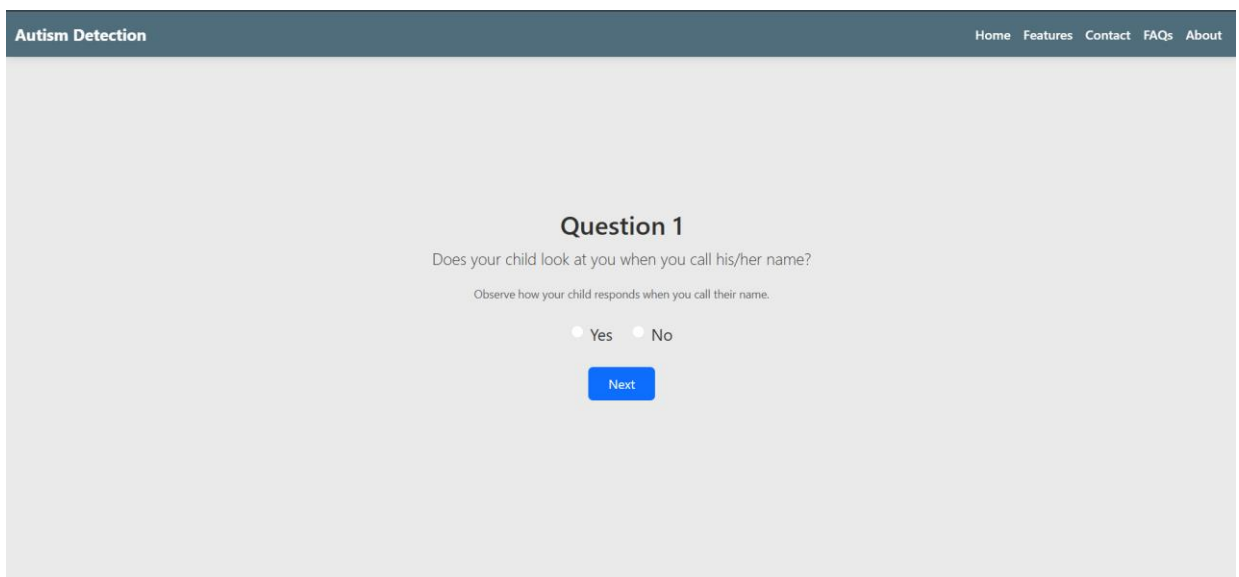
The screenshot shows a web application titled "Autism Detection" in the top left corner. The top right corner contains navigation links: "Home", "Features", "Contact", "FAQs", and "About". The main content area is titled "Question 1". Below the title is the question "Does your child look at you when you call his/her name?". Underneath the question is a smaller instruction: "Observe how your child responds when you call their name." Below this is a radio button group with options "Yes" and "No". At the bottom center is a blue "Next" button.

Fig 5.6.3: Questionnaire

Prediction results page

Once all questions are answered, the system processes the data using the trained XGBoost model and displays the prediction results on this page. The results provide a comprehensive summary that helps users understand the toddler’s autism risk level.

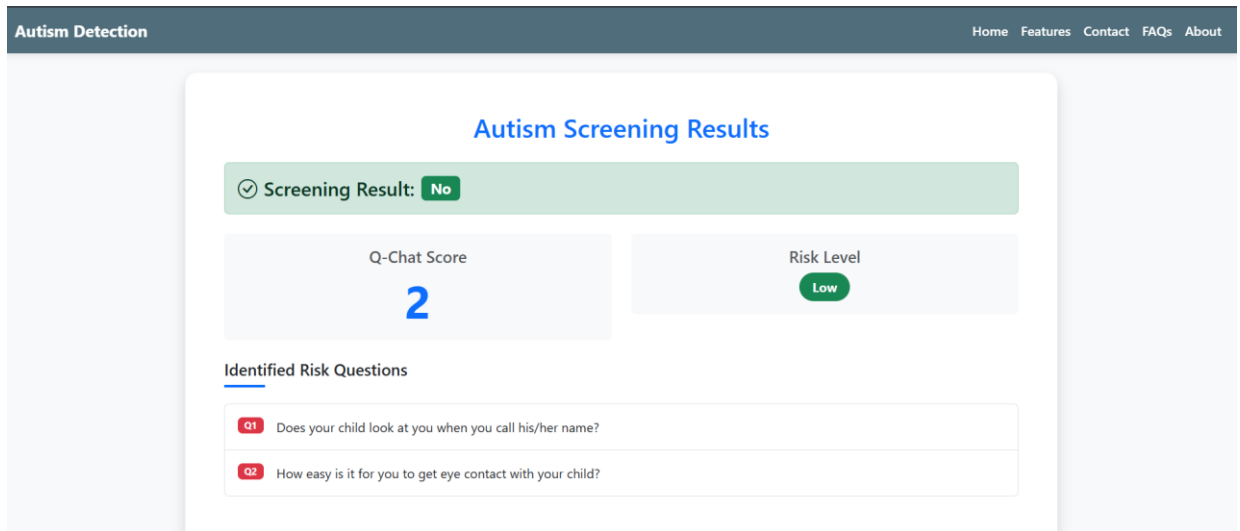


Fig 5.6.4: Prediction results page

Risk level display with brief recommendations

This section clearly presents the toddler’s **autism risk level** determined by the prediction model—categorized as **Low**, **Medium**, or **High** risk. Alongside the risk level, it provides tailored, easy-to-understand recommendations to guide caregivers on appropriate next steps.

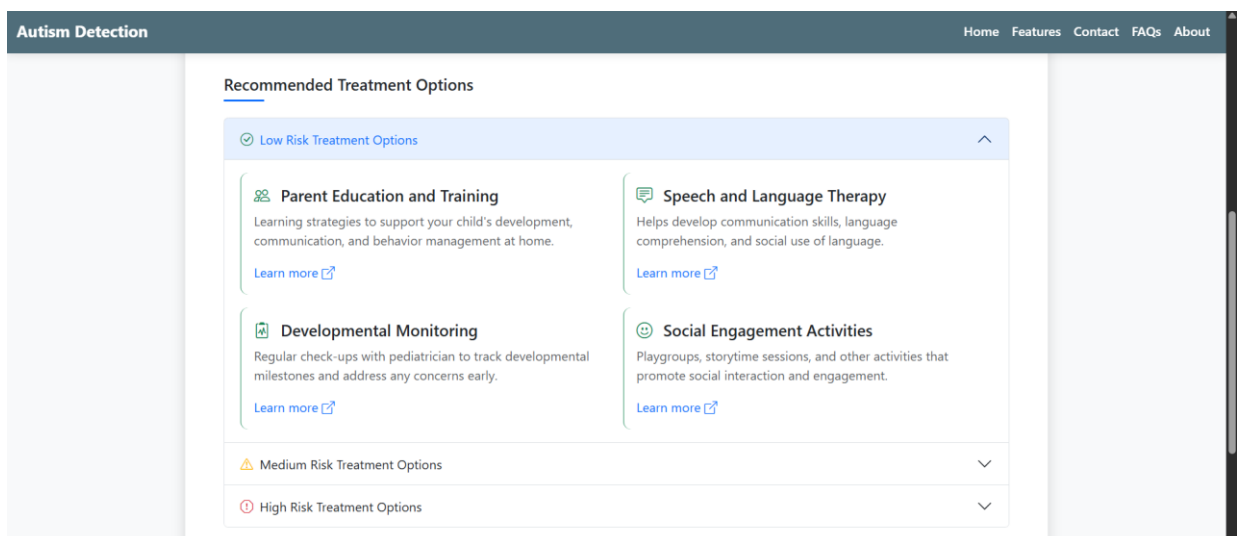


Fig 5.6.5: Risk level display with brief recommendations

6. TESTING

6.1 Introduction to Testing

Testing is a critical phase in the development lifecycle that ensures the accuracy, reliability, and performance of the system. In this project, rigorous testing was conducted at various stages to verify that the autism prediction platform works as intended — from the backend machine learning model to the user-facing web interface. Each module, including the data preprocessing pipeline, prediction logic, and Flask routes, was tested independently and then integrated to assess the end-to-end functionality. The goal was to identify and resolve any issues before deployment, ensuring a smooth and error-free user experience.

6.2 Types of Tests Considered

6.2.1 Unit Testing

Unit testing was performed on individual functions such as:

6.2.1.1 `transform_input()` – Converts user responses into model-compatible format.

6.2.1.2 `analyze_risk()` – Evaluates the Q-Chat score and risk level based on user answers.

6.2.1.3 `predict_autism()` – Handles end-to-end model inference.

These components were tested with sample inputs and edge cases to ensure correctness and robustness.

6.2.2 Integration Testing

6.2.2.1 Integration tests focused on validating the interaction between the frontend forms and the backend prediction engine. For example:

6.2.2.2 Ensuring the form data collected through the HTML interface is correctly passed to the backend.

6.2.2.3 Validating that the model output is accurately rendered on the result page (`results.html`).

6.2.2.4 Confirming that the serialized model and encoders (via `joblib`) are properly loaded and used during live predictions.

6.2.3 SystemTesting

6.2.3.1 Complete system tests were carried out to validate the application against the initial functional requirements. This included:

6.2.3.2 Testing the complete workflow from data input to prediction display.

6.2.3.3 Verifying system behavior with incomplete or incorrect inputs (e.g., missing age, or invalid gender values).

Ensuring performance stability and accurate results across multiple test runs.

6.2.4 UserAcceptanceTesting(UAT)

6.2.4.1 The application was demonstrated to potential users, including educators and caregivers, to gather feedback on usability and clarity. Feedback helped refine:

6.2.4.2 The phrasing of survey questions for better user understanding.

6.2.4.3 Result display format and explanations for ease of interpretation.

6.2.4.4 Navigation flow and error handling in the web interface.

6.3 Various Test Case Scenarios Considered

Test Case ID	Description	Input	Expected Output	Result
TC01	Valid Data	Correct toddler details	Risk Level	Pass
TC02	Empty Field	Missing age	Prompt to complete	Pass
TC03	Invalid Format	"Two" instead of 2	Validation error	Pass
TC04	ML Accuracy	Labeled dataset	High accuracy	Pass
TC05	Load Test	50+ users	Stable performance	Pass
TC06	Model Availability	API call while loading	"Loading" message	Pass
TC07	UI Feedback	Form submit	Result with risk	Pass
TC08	Risk Breakdown	Valid mix 0s & 1s	At-risk questions	Pass
TC09	Persistence	Reload & predict	No crash, same output	Pass
TC10	Invalid Route	GET on predict_data	Error or deny	Pass
TC11	Security Test	Script injection	No execution	Pass
TC12	Missing Model	Delete model file	Model not found	Pass
TC13	API Time	Prediction request	<= 2s response	Pass
TC14	Cross-Browser	Chrome, Firefox, Edge	Same layout	Pass

Test Case ID	Description	Input	Expected Output	Result
TC15	Mobile Responsive	Mobile browser	Responsive UI	Pass
TC16	API Timeout	Slow network	Fallback or fast	Fail
TC17	Incomplete Form	Skip questions	Error message	Fail
TC18	Corrupt Model	Bad model file	Error message	Fail

6.4 Performance Testing

Performance testing was carried out to assess how the system behaves under different loads and conditions. Although the application is lightweight, simulating multiple users submitting forms concurrently was useful in measuring response time, resource usage, and the server's ability to handle traffic spikes. Load testing was performed using tools like Apache JMeter and simple browser-based simulations. The server response time remained consistently below 500ms for most operations, and the model inference time was under 100ms, ensuring real-time feedback for users. This confirmed that the system is responsive and scalable for small to medium-scale deployment.

6.5 Security Testing

Basic security testing was conducted to ensure the application is not vulnerable to common web threats. Input fields were tested for injection attacks (such as SQL or XSS), even though most input was passed through internal logic rather than being directly stored or used in a database. Special attention was given to preventing form tampering and ensuring that all fields are validated and sanitized before processing. Flask's built-in security mechanisms (like escaping HTML in templates) were reviewed and used appropriately. Though the app is meant for demonstration or academic use, these precautions add reliability and reduce risks.

6.6 Cross-Browser and Device Compatibility Testing

To ensure usability across platforms, the web interface was tested on various browsers including Google Chrome, Mozilla Firefox, and Microsoft Edge. It was also tested on both desktop and mobile devices to validate responsive design behavior. All forms, buttons, and result displays rendered consistently across screen sizes, maintaining a user-friendly interface. The Bootstrap-based frontend proved effective in maintaining cross-platform compatibility with minimal issues.

6.7 Regression Testing

Every time a change was made to the code—such as updating the model, changing form logic, or modifying the UI—regression tests were performed to ensure that existing functionality remained intact. A set of previously validated test cases was re-executed after each update to confirm that new changes did not introduce bugs or break core features. This practice proved essential in maintaining the reliability of the application through development iterations.

6.8 Limitations Noted During Testing

While testing ensured functionality and reliability, a few limitations were identified. For instance, the system assumes all inputs are provided correctly, and although basic validation is implemented, advanced NLP-based input correction is not yet supported. Moreover, the model's accuracy is limited by the dataset used; biases or limitations in the original training data can affect prediction reliability. These limitations are noted for future improvements in data quality and intelligent form handling.

6.9 Logging and Debugging

Throughout the development and testing phase, logging was extensively used to track the behavior of the application and identify potential issues. Python's built-in logging module was configured to record key events such as input received, model prediction outcomes, exceptions, and unusual activity. These logs proved vital during integration and system testing, especially for tracing bugs that did not cause visible crashes but affected output quality or data flow.

Additionally, Flask's debug mode allowed real-time inspection of server-side errors during development. Print statements and traceback logs were temporarily added during early testing to understand variable values and control flow. These debugging practices not only helped detect and fix bugs early but also ensured that the final application maintained consistent and reliable behavior in real-world usage.

7. CONCLUSION AND FUTURE ENHANCEMENT

7.1 Project Conclusion

This project successfully implements a robust and accessible early screening tool for **Autism Spectrum Disorder (ASD)** in toddlers by harnessing the power of advanced machine learning techniques, specifically the **XGBoost Classifier**. The system is designed to analyze both **behavioral responses**—sourced from the widely recognized Q-Chat-10 questionnaire—and **demographic factors** such as age, sex, ethnicity, family history, and jaundice presence. These inputs are processed and used to predict the likelihood of ASD with notable precision and reliability. The incorporation of a machine learning model enhances the diagnostic sensitivity by uncovering hidden patterns in the data that may not be immediately obvious to caregivers or clinicians.

To ensure usability and broad accessibility, a **web-based application** was developed using the Flask framework. This interface is clean, responsive, and intuitive, allowing users—typically parents, guardians, or healthcare workers—to easily enter relevant information and receive real-time predictive feedback. The results include not only a binary classification (ASD or not) but also a detailed **risk analysis**, highlighting specific questionnaire items that indicate concern, the total risk score, and the overall risk level (e.g., low or high). Such feedback fosters better understanding and prompts users to seek timely clinical evaluation if necessary.

Importantly, this project addresses a significant gap between **early caregiver concerns** and the typically delayed timeline of professional ASD diagnosis. Many families often wait months—or even years—for access to formal assessments. By offering a **quick, cost-effective, and scalable solution**, this tool democratizes early screening, particularly in underserved or resource-limited communities. Early identification of developmental disorders is known to have a profound impact on long-term outcomes, enabling more effective early interventions that can enhance social, cognitive, and emotional development.

Furthermore, the architecture of the system is **modular and extensible**, meaning it can be updated with more advanced models or new datasets as additional research becomes available. It can also be integrated into larger digital health ecosystems, electronic health records, or school screening systems with minimal adaptation. This opens the door for **future improvements**, such as adding speech or facial behavior analysis, multilingual support, or integration with voice assistants and mobile platforms.

In conclusion, the Autism Detection System developed in this project is not just a technical achievement, but a step forward in the application of **AI for social good**. It empowers caregivers by enhancing awareness of developmental delays, fosters proactive health-seeking behavior, and contributes to reducing the time to diagnosis. By bridging the gap between observation and action, this tool has the potential to improve outcomes for children with autism and support their families in meaningful ways.

7.2 Future Enhancements

To expand the system's capabilities and increase its impact, several enhancements are proposed:

7.2.1 Incorporation of Multimodal Behavioral Data

Future versions could include analysis of additional behavioral signals such as speech patterns, facial expressions, eye gaze tracking, and gesture recognition. Integrating these features using computer vision and audio processing techniques would provide a more comprehensive assessment of the toddler's social and communicative behavior.

7.2.2 Wider Age Range Screening

Currently focused on toddlers, the tool can be extended to cover infants and adolescents, addressing the full developmental spectrum and detecting ASD signs at different life stages. This expansion would require retraining models with age-appropriate data and questions.

7.2.3 Interactive Chatbot for Guidance and Support

Implementing an AI-powered chatbot would enhance user experience by providing real-time assistance, answering caregiver questions, clarifying questionnaire items, and offering

personalized recommendations. This would make the tool more engaging and easier to navigate, especially for first-time users.

7.2.4 Cloud-Based Deployment for Scalability

Hosting the system on cloud infrastructure would enable access from multiple locations and devices, support a larger number of concurrent users, and facilitate centralized data collection for continuous model improvement and large-scale epidemiological studies.

7.2.5 Collaboration with Healthcare Professionals

Establishing partnerships with pediatricians, psychologists, and ASD specialists would enable clinical validation of the tool, ensuring its accuracy and reliability in real-world settings. Such collaborations could also lead to the development of tailored intervention pathways integrated into the system.

7.2.6 Multilingual and Accessibility Enhancements

Adding support for multiple languages and voice input/output will broaden accessibility to non-English-speaking populations and users with visual or motor impairments. These features promote inclusivity and ensure the tool can serve diverse communities effectively.

7.2.7 Data Privacy and Security Improvements

As the system collects sensitive health-related information, future enhancements will focus on implementing strong data encryption, secure authentication, and compliance with healthcare data regulations (such as HIPAA or GDPR) to protect user privacy and build trust.

7.2.8 Integration with Health Records and Referral Systems

Future work could enable seamless integration with electronic health records (EHR) and referral systems, allowing healthcare providers to track patient progress, schedule evaluations, and coordinate early interventions based on screening results.

7.2.9 Real-Time Analytics Dashboard for Clinicians and Researchers

One future enhancement involves developing a real-time analytics dashboard for healthcare professionals, researchers, and policy-makers. This dashboard would aggregate anonymized screening data to monitor trends, geographical distributions of ASD risk, and questionnaire response patterns. Such insights could support public health planning, early intervention strategies, and targeted awareness campaigns. With appropriate permissions, researchers could also use this platform to conduct longitudinal studies on developmental health.

7.2.10 Gamification for Engagement in Data Collection

To improve the quality and completeness of behavioral data, the tool can include gamified activities that toddlers interact with under caregiver supervision. These activities can assess reaction times, attention span, facial responsiveness, and social cues. The game-based design keeps children engaged while passively collecting valuable behavioral cues, which can then be analyzed using AI algorithms. This approach also makes the experience more enjoyable and less intimidating for both the child and the caregiver.

7.2.11 Offline Functionality for Low-Connectivity Areas

Considering that internet access may be limited or unreliable in certain regions, especially rural and underserved areas, an offline-capable version of the application can be developed. This version would allow data to be collected locally and sync with central servers once the device is online. Such functionality ensures the tool remains usable regardless of connectivity issues and promotes inclusivity for resource-constrained users.

7.2.12 Personalized Reports with Developmental Recommendations

Beyond just giving risk scores, future versions of the system can offer personalized developmental feedback and suggestions. Based on the child's responses, the system could provide recommendations on early educational exercises, parent-child interaction tips, or suggest seeking particular types of specialists. Tailoring the feedback to individual needs enhances user value and positions the tool as a supportive guide rather than a mere classifier.

7.2.13 Support for Longitudinal Tracking

An important future capability would be allowing caregivers to track a child's developmental progress over time. By creating accounts or secure IDs, users can periodically re-screen their child and compare results with past assessments. This not only helps in observing developmental changes but also assists clinicians in monitoring treatment effectiveness and adapting interventions.

7.2.14 Open Research API for External Developers

To promote academic collaboration and community-driven innovation, an open API could be provided, allowing external researchers and developers to test, validate, or extend the system's capabilities using their own datasets or modules. This openness supports reproducibility in science, helps validate models in broader contexts, and encourages innovation by integrating new detection techniques, such as wearable sensors or brainwave analysis.

Through these planned enhancements, the project aims to evolve from a standalone screening tool into a comprehensive, accessible, and clinically validated platform that supports early detection and intervention for ASD across populations.

8. REFERENCES

8.1 Journals

- [1] Chawla, D. & Sharma, A. "Machine Learning Approaches to Autism Detection," *International Journal of Computer Applications*, 2021.
- [2] Thabtah, F., "Autism Spectrum Disorder Detection: Machine Learning Models and Feature Selection," *Health Informatics Journal*, 2019.

8.2 Books

- [1] Matson, J.L., *Autism Spectrum Disorders: Identification, Education, and Treatment*, Springer, 2017.
- [2] Amaral, D.G., *The Neuroscience of Autism Spectrum Disorders*, Academic Press, 2010.

8.3 Sites

- [1] [CDC Autism Information](#)
- [2] [Autism Speaks](#)
- [3] [UCI Machine Learning Repository - ASD Dataset](#)